Crime Scene VR — Colour Blind Modes: Research & Development Report

Project: Crime Scene (Unity 6000.4.9f1, URP 17.4, BNG VR Interaction Framework, OpenXR / Meta Quest 3S) **Branch:** TabletSettingsGrabScriptZfightingFixDoorsScriptTabletRecording **Date:** 2026-07-17 **Scope:** What colour vision deficiency actually is, which modes are worth shipping, the algorithms and matrices behind them, how to implement them in this project's URP + VR stack, and what the filters cannot fix. **Status:** Research only — no code changed. `SettingsManager.SetColorblindMode()` remains stubbed.

---

1. Executive Summary

The project already reserves a slot for this feature: `settings.json` carries `accessibility.colorblindMode`, and `SettingsManager.SetColorblindMode(string)` persists it behind a `// TODO: Colorblind mode coming in a separate issue.` comment (`SettingsManager.cs:209`). This report defines what should go behind that stub.

Five findings drive the recommendation:

1. **"Colour blind mode" is ambiguous, and getting it backwards is the classic failure.** Applying a *simulation* matrix (the thing that shows a normal-sighted developer what protanopia looks like) to a colour blind player's view makes their vision **worse**, not better. The accessibility feature is *correction* — **daltonization** — which is a different matrix. The menu label "Protanopia" must mean "*I have protanopia, compensate for me*", not "*simulate protanopia*". Both are worth building, for different audiences: correction for players, simulation for the team's own testing.
2. **Daltonization collapses to a single 3×3 matrix — verified numerically in this study.** The textbook algorithm is described as a five-stage pipeline (linearise → LMS → simulate → compute error → redistribute). Because every stage except the final clamp is linear, the whole chain algebraically reduces to one matrix `D = I + S(I − M)`. Tested over 200,000 random colours, the collapsed matrix matches the stepwise pipeline to **4.4×10^{-16}** — floating-point rounding. This makes the runtime cost of the feature one `mul` per pixel, and it means the effect can be shipped **with no custom shader at all** (see §7.2).
3. **The widely-copied tritanopia matrix is broken, and this study quantifies how badly.** The naive LMS projection used by most blog-post implementations pushes **73.1%** of the RGB cube out of gamut (channel range −2.61 … 3.61) — pure red maps to a blue channel of −3.011. Building the same correction on the Machado 2009 matrices instead drops this to **12.5%** clipping (range −0.41 … 1.41). **Use Machado; do not copy the daltonize.org tritan matrix.**
4. **This project's tablet UI is already colour blind safe — the risk is in the world, not the interface.** An audit of the actual shipping `TabletTheme` palette under simulated protanopia, deuteranopia and tritanopia (§8.4) found that every meaning-carrying colour pair retains large perceptual separation; the Accent/Amber pair keeps ΔE 73–144 across all three. The one weak pair (`Surface` vs `Background`, ΔE 7) is weak for **normal vision too** — it is a luminance problem, not a colour vision one. The team should therefore spend its effort on the **evidence-finding loop in the 3D scene**, not on re-theming the iPad.
5. **A naive implementation will corrupt the photo gallery.** A URP Renderer Feature applies to *every* camera on that renderer — including `IpadCamera`'s capture camera. Photos would be daltonized when written to

the `Texture2D`, then daltonized **a second time** when the gallery displays them through the world-space canvas. The capture camera must be excluded (§8.2). This is the single most likely bug in this feature.

Recommendation: ship a four-option correction mode (Off / Protan / Deutan / Tritan) with a severity slider, implemented as a Full Screen Pass Renderer Feature, excluded from the capture camera, plus a separate developer-only simulation toggle. Treat the filter as a **secondary** measure — the primary fix for the evidence loop is redundant, non-colour cues (§9). Phasing in §11.

---

2. Why This Matters Here

The core loop is: *walk the scene, spot key evidence, photograph it, get graded*. Detection is the game. `KeyEvidenceItem` marks objects and `IpadCamera.DetectAndMarkEvidence()` grades what was in frame — but none of that helps a player who **cannot see the evidence in the first place**.

That is the specific risk. Crime scene evidence is disproportionately red-on-organic: bloodstains on carpet, wood, and soil. A red stain on a brown rug is close to the textbook worst case for protanopia and deuteranopia, which is roughly **1 in 12 men (8%)** and 1 in 200 women. For a graded experience this is not cosmetic — it changes the score. A protanope failing the grade because they physically could not see a bloodstain is a correctness bug in the assessment, not a preference issue.

Two existing settings make this sharper:

- **Night mode** (`SettingsApplier.ApplyTimeOfDay`) drops the scene to a flat dark ambient (`nightAmbientColor` ≈ RGB 0.05, 0.07, 0.13). Colour discrimination degrades for *everyone* in low light as vision shifts toward rod-dominated scotopic response, and it compounds an existing deficiency. Night + protanopia + a dark red stain is the hardest combination the game can produce.
- **Grading is pass/fail and not persisted** (`EvidenceGradingManager`), so a player cannot recover a missed item after the fact.

---

3. The Biology, Briefly

Normal human colour vision is **trichromatic**: three cone photoreceptor classes, each with a different spectral sensitivity peak.

Cone	Name	Peak sensitivity	Informally
L	Long-wavelength	~564–580 nm	"red"
M	Medium-wavelength	~534–545 nm	"green"
S	Short-wavelength	~420–440 nm	"blue"

Colour is not wavelength — it is the **ratio** of the three cone responses. Any condition that removes a cone class, or shifts its sensitivity curve toward a neighbour, collapses part of that ratio space. Information is lost at the retina; no display trick can restore it. What a "colour blind mode" can do is **re-encode** information from a lost axis onto an axis the viewer still has.

That framing explains everything downstream:

- **L and M curves overlap heavily** (~564 vs ~534 nm — barely 30 nm apart). Losing either collapses the red–green axis, which is why protanopia and deuteranopia are both "red-green" deficiencies and why they are so common.
- **S sits far away** (~420–440 nm), so losing it collapses the blue–yellow axis instead — a different, rarer failure.
- **The L and M genes both sit on the X chromosome.** Males have one X, so a single defective copy is uncorrected: red-green deficiency is ~8% of males but ~0.5% of females. **The S-cone gene is on chromosome 7**, an autosome — which is why tritanopia affects both sexes equally.

---

4. Taxonomy — The Modes That Matter

Three severity tiers, not two. Most implementations only handle the middle one.

4.1 Anomalous trichromacy — all three cones present, one shifted

The **most common** form by far, and the one most often ignored. Vision is still trichromatic but compressed; severity is a continuum, not a switch.

Type	Cone affected	Prevalence (males)	Notes
Protanomaly	L shifted toward M	~1%	Reds dimmed and desaturated
Deuteranomaly	M shifted toward L	**~5%**	**The single most common form of CVD**
Tritanomaly	S shifted	very rare (<0.01%)	Blue-yellow compression

> **Design consequence:** a mode that only offers full dichromat correction over-corrects the ~5% of men with mild deuteranomaly, who are the largest group. This is the argument for a **severity slider**, and it is exactly what the Machado 2009 model parameterises (§6.3).

4.2 Dichromacy — one cone class entirely absent

Type	Missing cone	Prevalence	Confusion axis
Protanopia	L ("red")	~1.0–1.5% of males (≈0.51% of population)	Red–green; **reds appear very dark, near black**
Deuteranopia	M ("green")	~1.0–1.5% of males (≈0.64% of population)	Red–green; luminance largely preserved
Tritanopia	S ("blue")	~0.01%, sexes equal	Blue–yellow

The protanopia/deuteranopia distinction matters for this project: **protanopes lose red luminance**, so a dark red bloodstain doesn't just shift hue — it goes *darker*, sinking toward the background. Deuteranopes keep roughly correct brightness and mainly lose the hue distinction. A red-on-brown stain therefore fails **differently** for the two, and a protan-specific check is worth doing.

4.3 Monochromacy — no usable colour discrimination

Type	Prevalence	Notes
Achromatopsia (rod monochromacy)	~1 in 30,000	No functional cones; usually with photophobia and low acuity
Blue-cone monochromacy	~1 in 100,000	Only S cones; X-linked

> **Important and often missed:** daltonization **cannot help achromatopsia**. There is no surviving colour axis to redistribute information onto. The only useful measures are **luminance contrast, shape, and text**. Offering an "Achromatopsia mode" that applies a colour matrix is theatre. If the team wants to serve these players, the answer is a high-contrast mode and the redundant cues in §9 — not a filter.

Also worth stating plainly: **CVD is not always inherited.** Cataracts, glaucoma, diabetic retinopathy, multiple sclerosis and some medications cause *acquired* deficiency — and acquired forms skew tritan. Cataracts alone are more common than inherited colour blindness. The audience is wider than the 8% figure suggests.

---

5. The Critical Distinction: Simulation vs Correction

This is the conceptual core of the report, and the thing most likely to be got wrong.

	Simulation	**Correction (Daltonization)**
Question answered	"What does a protanope see?"	"How do I help a protanope see?"
Audience	Normal-sighted developers, QA	Colour blind players
Effect on a CVD player	**Makes it worse** — deficiency applied twice	Recovers lost distinctions
Matrix	`M` (Machado / Viénot)	`D = I + S(I − M)`
Where it belongs	Dev menu, editor tooling, screenshots	The player-facing Settings app

The trap is that a simulation matrix *looks* like a colour blind mode when a normal-sighted developer switches it on — the screen visibly changes, the reds go muddy, and it feels like the feature works. It does the exact opposite of the intended job.

Naming matters. A settings entry reading `Protanopia` must mean "*I am a protanope — correct the image for me.*" This is the convention players expect: Forza Horizon 4, Call of Duty and most shipped titles label the option by the player's condition, not by the transform applied. The stored value in `settings.json` should follow suit.

Both are worth having. The team needs simulation to check its own scene dressing — is that bloodstain visible on that carpet? — and players need correction. They should be separate switches: correction in the tablet's Settings app, simulation in a dev-only menu that never ships to players (this project already has `Scripts/Dev/` and `Scenes/Dev/` for exactly this kind of thing).

> Microsoft's guidance is explicit that simulation filters are an internal development tool and **must not** replace testing with actual colour blind players.

---

6. The Algorithms

6.1 Colour space correctness — get this right first

Every matrix below is defined on **linear RGB**. Applying them to gamma-encoded sRGB values is the second most common implementation error after simulate/correct confusion, and it produces subtly wrong, muddy results that still look plausible enough to ship.

```
sRGB → linear:   c ≤ 0.04045 ? c/12.92 : ((c + 0.055)/1.055)^2.4
linear → sRGB:   c ≤ 0.0031308 ? c*12.92 : 1.055 * c^(1/2.4) − 0.055
```

Unity specifics that make this mostly free here:

- This project renders in **Linear** colour space (the URP default, and effectively mandatory for URP). Inside a Renderer Feature at `AfterRenderingPostProcessing`, the camera colour buffer is **already linear** — so no manual conversion is needed. Just multiply.
- If the pass is instead placed after the final sRGB conversion, or the effect is done in a UI shader on gamma values, the conversion must be done by hand.
- **Do not** convert to linear, multiply, and convert back inside a pass that is already linear. That double-converts and washes the image out.

6.2 Viénot, Brettel & Mollon (1999) — the classic

The long-standing standard, and the basis of most game implementations. Convert linear RGB → LMS, project onto the plane of colours the dichromat can still distinguish, convert back.

The RGB→LMS matrix used by the reference daltonize.org implementation:

```
        [ 17.8824    43.5161     4.11935  ]
RGB2LMS=[  3.45565   27.1554     3.86714  ]
        [  0.0299566  0.184309   1.46709  ]
```

LMS-space projections:

```
Protanopia:    L' = 0·L + 2.02344·M − 2.52581·S     (M, S unchanged)
Deuteranopia:  M' = 0.494207·L + 0·M + 1.24827·S    (L, S unchanged)
Tritanopia:    S' = −0.395913·L + 0.801109·M + 0·S  (L, M unchanged)
```

Collapsed to linear RGB (`Sim = LMS2RGB · P · RGB2LMS`), computed in this study:

```
Protanopia          Deuteranopia         Tritanopia
[ 0.1124  0.8876  0][ 0.2928  0.7073  0][ 0.4933  0.5067   0]
[ 0.1124  0.8876  0][ 0.2927  0.7072  0][ 0.4933  0.5067   0]
[ 0.0040 −0.0040  1][−0.0223  0.0223  1][−3.0109  3.0109   1]
```

Note the sanity check baked into the first two: rows 1 and 2 are **identical**, meaning R and G collapse onto a single axis — exactly what dichromacy means. The structure confirms the maths is right.

Now look at the tritanopia matrix: **−3.0109**. Pure red maps to a blue channel of −3.011, three times outside the gamut. That is not a typo — it is a known defect of the naive tritan projection, which makes an incorrect assumption about the anchor colour for blue. §6.4 quantifies the damage.

Brettel, Viénot & Mollon (1997) is the more rigorous ancestor: it projects onto **two** half-planes joined along a neutral axis rather than one plane, which handles tritanopia correctly. Viénot 1999 is the cheaper single-plane simplification that is accurate for protan/deutan but *not* for tritan. Most implementations copy the simplification and then apply it to all three types — which is precisely the bug above.

6.3 Machado, Oliveira & Fernandes (2009) — the modern choice

Published in *IEEE TVCG* 15(6):1291–1298. A physiologically-based model grounded in the **stage theory** of colour vision and derived from electrophysiological data. Its advantages over Viénot for this project:

- **Unified.** Handles normal vision, anomalous trichromacy *and* dichromacy in one model — Viénot only does dichromacy.
- **Severity-parameterised.** A continuous 0.0 → 1.0 parameter models the cone sensitivity *shift*, not just its removal. Severity 0 is identity; severity 1 is full dichromacy. This directly serves the ~5% deuteranomalous population that a dichromat-only filter over-corrects.
- **Validated** experimentally against both CVD and normal-vision participants.
- **Correct for tritan**, unlike the naive projection.

Matrices at **severity 1.0** (full dichromacy), operating on linear RGB:

```
Protanomaly (1.0)                    Deuteranomaly (1.0)                  Tritanomaly (1.0)
[ 0.152286  1.052583  -0.204868 ]    [ 0.367322  0.860646  -0.227968 ]    [ 1.255528  -0.076749
-0.178779 ]
[ 0.114503  0.786281   0.099216 ]    [ 0.280085  0.672501   0.047413 ]    [-0.078411   0.930809
0.147602 ]
[-0.003882 -0.048116   1.051998 ]    [-0.011820  0.042940   0.968881 ]    [ 0.004733   0.691367
0.303900 ]
```

At **severity 0.5** (the realistic anomalous-trichromat case):

```
Protanomaly (0.5)                    Deuteranomaly (0.5)                  Tritanomaly (0.5)
[ 0.458064  0.679578  -0.137642 ]    [ 0.547494  0.607765  -0.155259 ]    [ 1.017277  0.027029
-0.044306 ]
[ 0.092785  0.846313   0.060902 ]    [ 0.181692  0.781742   0.036566 ]    [-0.006113  0.958479
0.047634 ]
[-0.007494 -0.016807   1.024301 ]    [-0.010410  0.027275   0.983136 ]    [ 0.006379  0.248708
0.744913 ]
```

Severity 0.0 is the identity matrix for all three. Intermediate severities are obtained by **linear interpolation between the published matrices** — this is what the reference `colour-science` implementation does, and it is cheap enough to do per-frame on the CPU and push as a shader uniform. Interpolating the matrix (not the result) means the severity slider costs nothing at pixel rate.

6.4 Daltonization — the correction, and two findings

The standard LMS daltonization algorithm:

1. Linearise the sRGB input.
2. Simulate the deficiency: `sim = M · rgb`.
3. Compute the **error** — the information the viewer loses: `err = rgb − sim`.
4. **Redistribute** that error into channels the viewer *can* still discriminate: `corr = S · err`.
5. Add it back and clamp: `out = clamp(rgb + corr)`.
6. Re-encode to sRGB.

The redistribution matrix `S` for red-green deficiency (the classic values) pushes the lost red-green error into the green and blue channels — turning an invisible hue difference into a visible blue/yellow one, along the axis a protan/deutan viewer has fully intact:

```
      [ 0    0    0 ]
S_rg =[ 0.7  1    0 ]
      [ 0.7  0    1 ]
```

Finding 1 — the pipeline is one matrix

Steps 2–5 are all linear, so they compose:

```
out = rgb + S(rgb − M·rgb)
    = rgb + S(I − M)·rgb
    = (I + S(I − M))·rgb
    = D · rgb
```

Verified: across 200,000 random colours, the collapsed `D` matched the literal stepwise pipeline to a maximum absolute error of **4.4×10^{-16}** — i.e. exactly, to double-precision rounding. (The final clamp is nonlinear but happens once at the end either way, so it rides outside the collapse.)

Consequences, and they are large:

- Runtime cost is **one 3×3 multiply per pixel** — a single `mul` in HLSL. Not a pipeline, not multiple passes.
- Because the whole effect is a colour matrix, it can be expressed as a **URP Channel Mixer volume override with zero custom shader code** (§7.2).
- Severity blending stays free: interpolate `M` on the CPU, rebuild `D`, upload.

Daltonization matrices built on **Machado severity 1.0**, ready to ship:

```
Protan D                          Deutan D                          Tritan D
[ 1.0000   0.0000   0.0000 ]      [ 1.0000   0.0000   0.0000 ]      [ 0.7412  -0.4072
0.6660 ]
[ 0.4789   0.4769   0.0442 ]      [ 0.1628   0.7250   0.1122 ]      [ 0.0751   0.5852
0.3397 ]
[ 0.5973  -0.6887   1.0914 ]      [ 0.4547  -0.6454   1.1907 ]      [ 0.0000   0.0000
1.0000 ]
```

Finding 2 — the naive tritan correction is unusable; Machado's is fine

Sampling a 32^3 grid over the RGB cube and measuring how much of it leaves the displayable [0,1] range:

Tritan daltonization built on	% of RGB cube out of gamut	Channel range
Naive LMS projection (daltonize.org)	**73.1%**	−2.614 … 3.614
Machado 2009	**12.5%**	−0.407 … 1.407

Nearly three quarters of all colours clip with the naive matrix — the correction is destroying the image, not improving it. This is a quantified confirmation of the known warning that most tools simulate tritanopia incorrectly. **Use the Machado-derived matrices.**

Finding 3 — clipping is significant even when correct, so plan for it

Out-of-gamut behaviour of the *correct* matrices:

Matrix	% of cube clipping	Range
Machado protan **simulation** (1.0)	17.2%	−0.205 … 1.205
Machado deutan **simulation** (1.0)	7.6%	−0.228 … 1.228
Machado tritan **simulation** (1.0)	24.3%	−0.256 … 1.256
Machado protan **daltonization**	25.8%	−0.689 … 1.689
Machado deutan **daltonization**	26.7%	−0.645 … 1.645
Machado tritan **daltonization**	12.5%	−0.407 … 1.407
Machado protan simulation @ **severity 0.5**	8.8%	−0.138 … 1.138
Machado deutan simulation @ **severity 0.5**	4.4%	−0.155 … 1.155

Two things follow. First, ~26% of colours clip under a full-strength correction — a naive `saturate()` will flatten detail in saturated regions, and this is exactly where the evidence (red stains) lives. Worth evaluating a soft-rolloff or luminance-preserving clamp rather than a hard clamp, and worth exposing the severity slider so players can back off. Second, severity 0.5 roughly **halves** the clipping — another argument for not defaulting to full strength.

---

7. Implementation Options in Unity 6 / URP 17.4

Four viable routes, in rough order of recommendation.

7.1 Option A — Full Screen Pass Renderer Feature ★ recommended

URP's supported path for custom post-processing since URP 14, and current in URP 17.

Setup:

1. `Create > Shader Graph > URP > Fullscreen Shader Graph` (or a hand-written HLSL `Blit`-style pass).
2. Sample `URP Sample Buffer → BlitSource`, multiply by the 3×3 matrix exposed as a `float3x3` / three `float4` uniforms.
3. Create a Material from it.
4. On **both** `Assets/Settings/PC_Renderer.asset` and `Assets/Settings/Mobile_Renderer.asset`: `Add Renderer Feature > Full Screen Pass Renderer Feature`.
5. Set **Pass Material** to the material, **Injection Point** to `After Rendering Post Processing`, **Requirements** to `Color`.

Pros: full control; one pass; correct in linear space at that injection point; works with XR single-pass instanced so both eyes get an identical transform for free. **Cons:** costs a full-screen pass (see §10); must be added to both renderer assets; needs the capture-camera exclusion of §8.2.

> **Both renderers matter.** This project ships `PC_Renderer` and `Mobile_Renderer`. A feature added to only one produces the classic "works in Editor, missing on Quest" report.

7.2 Option B — Channel Mixer volume override ★ zero-code, best perf

This falls directly out of Finding 1. A URP **Channel Mixer** override *is* a 3×3 matrix on RGB (Red-Red, Red-Green, Red-Blue, …). Since daltonization collapses to exactly one 3×3 matrix, the entire feature can be expressed as a Channel Mixer with no shader written at all.

Better still: Channel Mixer is folded into URP's **colour grading LUT bake** (`LutBuilder3D`), which runs in the *existing* uber post-processing pass. If post-processing is already enabled, **the filter is effectively free** — no extra full-screen pass, no extra bandwidth. On a tile-based mobile GPU that is a decisive advantage.

Pros: no custom shader; no extra pass; runtime-settable via a scripted `VolumeProfile`; correct linear space automatically. **Cons:** requires post-processing enabled on the camera and a global Volume (if PP is currently off for perf, enabling it costs the uber pass — measure); applies to every camera using that volume, so the capture-camera problem of §8.2 still needs solving; less obvious to a reader than a named feature.

Verdict: if the project already runs post-processing on Quest, this is the cheapest correct implementation and should be tried **first**. If not, Option A.

7.3 Option C — Colour Lookup (LUT) texture

Author a 32×32×32 strip LUT per mode and feed it through the `Color Lookup` volume override.

Pros: also folds into the uber pass; can encode non-linear corrections a 3×3 cannot. **Cons:** pointless here — the transform *is* linear, so a LUT only adds interpolation error and texture memory over Option B. Only justified if the correction later becomes non-linear (e.g. gamut-aware rolloff).

7.4 Option D — Per-material / UI theme swap

Recolour assets and UI directly rather than filtering the frame.

Pros: zero per-pixel cost; can be *smarter* than a global filter (change a hue that a filter cannot distinguish anyway); `TabletTheme` is already a single source of truth, so a themed swap is genuinely cheap to implement here. **Cons:** doesn't touch the 3D world, textures, or lighting — which §8.4 shows is where this project's actual risk is. Complementary, not a substitute.

7.5 Comparison

	A: Full Screen Pass	B: Channel Mixer	C: LUT	D: Theme swap
Custom shader	Yes	**No**	No	No
Extra full-screen pass	**Yes**	No (folds into uber)	No (folds into uber)	No
Quest cost	~1–2.5 ms	~free if PP on	~free if PP on	Zero
Covers 3D world	Yes	Yes	Yes	**No**
Severity slider	Easy	Easy	Rebake needed	N/A
Capture-cam risk (§8.2)	Yes	Yes	Yes	No

---

8. Project-Specific Integration

8.1 The stub is already there

```
// SettingsManager.cs:209
public void SetColorblindMode(string mode)
{
    CurrentSettings.accessibility.colorblindMode = mode;
    // TODO: Colorblind mode coming in a separate issue.
    SaveSettings("iPad");
}
```

The plumbing is done: `AccessibilitySettings.colorblindMode` defaults to `"none"`, persists to `settings.json`, and `SaveSettings` raises `OnSettingsChanged`. The architecture the rest of the settings follow is: `SettingsManager` **owns data,** `SettingsApplier` **pushes it into the world.** Colour blind mode must follow the same split — `SetColorblindMode` should not touch a renderer directly.

Proposed additions, matching existing conventions:

```
"accessibility": {
  "subtitles": false,
  "colorblindMode": "none",        // none | protan | deutan | tritan
  "colorblindSeverity": 1.0,       // 0..1, Machado severity  (new)
  "colorblindSimulate": false      // DEV ONLY — simulate instead of correct (new)
}
```

`JsonUtility` will default new fields on an old `settings.json` without throwing, so this is backward compatible with saves already on disk. Validate `mode` against the four legal strings and log a warning on anything else, mirroring `SetMovementSpeed`'s existing validation.

Applying it belongs in `SettingsApplier`, which already re-resolves and re-applies on every `sceneLoaded` — the right place, since the filter must survive the `CSHouse ⇔ CS_Outside` teleport that the tablet persists across.

8.2 ⚠ The capture camera trap — the most likely bug

`IpadCamera` mounts a **second Camera** that renders continuously into a `RenderTexture` (`IpadCamera.cs:155–209`), which `CapturePhoto()` reads back into a `Texture2D` and stores in `PhotoLibrary`.

A Renderer Feature runs on **every camera using that renderer**. So by default:

1. The capture camera renders → **filter applied** → photo saved with daltonization **baked into the pixels**.
2. The gallery displays that photo on the tablet's **world-space** canvas.
3. The main VR camera renders that canvas → **filter applied again**.

The photo is daltonized **twice**. The correction is not idempotent — `D·D ≠ D` — so gallery photos come out visibly wrong (over-shifted, heavily clipped, and worse the higher the severity), while the live viewfinder next to them looks right. That is a confusing bug to diagnose from a Console log, because nothing errors.

Photos must be stored unfiltered. They are the ground truth of what the player photographed, and the filter is a *display* concern. Storing raw also means toggling the mode later correctly re-filters existing photos at display time instead of leaving a library of permanently-baked images.

Three ways to exclude the capture camera, best first:

1. **Give the capture camera its own Renderer asset.** Duplicate the renderer without the colour blind feature, and assign it on the capture camera's `UniversalAdditionalCameraData → Renderer`. Explicit, free, no code.
2. **Skip by target texture** in `AddRenderPasses`: the capture camera is the only one with `camera.targetTexture != null`. Concise, but an implicit rule a future reader can break.
3. **Skip by reference/tag** — hold a reference to the capture camera and compare. Awkward across scene loads for a persistent prefab.

Option 1 is the recommendation — it matches the project's stated preference for explicit setup over implicit rules, and it costs nothing at runtime.

> Same trap, same fix, for Option B: a global Volume affects the capture camera too. Either give it its own volume mask (culling `layerMask` on the camera), or disable post-processing on the capture camera outright.

8.3 What the filter does and does not reach

- **World-space UI is filtered — good.** The tablet screen is a **World-Space** canvas (`IpadCamera.cs:18–21` documents this: it is on the UI layer and is stripped from the capture camera precisely because it is in-world geometry). Because it renders as scene geometry through the main camera, a full-screen pass at `AfterRenderingPostProcessing` **does** cover it. The tablet UI gets corrected for free.
- **Screen-space *Overlay* canvases are not.** Overlay canvases composite after everything, outside the pass. This project's tablet is world-space so it is fine, but any future overlay HUD, fade, or loading screen would silently skip the filter.
- **Both eyes, identically.** A full-screen pass under XR single-pass instanced applies the same matrix to both eyes. This matters more in VR than on a flat screen: a per-eye colour mismatch causes **binocular rivalry** — genuine discomfort, not just an artefact. Do not hand-roll anything that could diverge per eye.

8.4 Audit — does this project's palette actually need help?

`TabletTheme` is the single source of truth for the tablet UI. Its colours were simulated under Machado severity 1.0 for each deficiency and measured by WCAG contrast ratio and CIE76 ΔE:

Theme colours as seen (hex):

Colour	Normal	Protanopia	Deuteranopia	Tritanopia
Background	`#1E1E20`	`#1D1E20`	`#1D1E20`	`#1D1E1E`
Surface	`#2C2C2F`	`#2C2C2F`	`#2B2C2F`	`#2C2D2D`
SurfaceRaised	`#48484B`	`#47484B`	`#47484B`	`#474849`
Accent	`#0A85FF`	`#3991FF`	`#007DFD`	`#00A3B6`
AccentSoft	`#1A528F`	`#335691`	`#1E4C8E`	`#006069`
TextPrimary	`#F2F2F5`	`#F2F3F5`	`#F2F2F5`	`#F2F3F3`
TextSecondary	`#9E9EA3`	`#9D9FA3`	`#9D9EA3`	`#9D9FA0`
Amber	`#FFB840`	`#D2BB2D`	`#E3CC44`	`#FFA4A0`

Separation of the pairs that carry meaning (contrast ratio / ΔE):

Pair	Normal	Protanopia	Deuteranopia	Tritanopia
Accent vs Amber — *selected state vs evidence marker*	2.10:1 ΔE138	1.63:1 ΔE133	2.43:1 ΔE144	1.61:1 **ΔE73**
Accent vs SurfaceRaised — *selected vs unselected button*	2.52:1 ΔE73	2.89:1 ΔE68	2.32:1 ΔE76	2.99:1 ΔE46
Accent vs AccentSoft — *active row vs playing row*	2.20:1 ΔE38	2.30:1 ΔE35	2.16:1 ΔE38	2.39:1 ΔE27
Amber vs TextPrimary	1.55:1 ΔE72	1.74:1 ΔE73	1.45:1 ΔE70	1.70:1 ΔE42
TextPrimary vs Background	14.95:1 ΔE84	14.95:1 ΔE84	14.95:1 ΔE84	14.95:1 ΔE84
TextSecondary vs Background	6.26:1 ΔE54	6.27:1 ΔE54	6.26:1 ΔE54	6.26:1 ΔE54
Surface vs Background	1.20:1 **ΔE7**	1.20:1 **ΔE7**	1.20:1 **ΔE7**	1.20:1 **ΔE7**

Conclusion: the tablet UI is already colour blind safe, essentially by accident of good design. No meaning-carrying pair drops below ΔE 27, and the worst case (Accent vs Amber under tritanopia, ΔE 138 → 73) still leaves an enormous margin — ΔE ≈ 2.3 is the *just-noticeable* threshold. The reason is structural and worth preserving deliberately:

- The scheme is **dark neutral + one accent**, so most distinctions are carried by **luminance**, which no colour matrix touches. Note the text rows are literally identical across all four columns.
- The two chromatic colours are **blue (Accent) and amber (Amber)** — opposite ends of the blue-yellow axis, which protanopia and deuteranopia leave **fully intact**. This is the single best two-colour choice for the 99% of CVD that is red-green, and the theme happens to have made it.
- Tritanopia is the only type that attacks the blue-yellow axis, and even there ΔE 73 survives — visible in the table as Amber shifting `#FFB840 → #FFA4A0` (amber → salmon) while Accent goes `#0A85FF → #00A3B6` (blue → teal). They separate.

The one genuinely weak pair, `Surface` vs `Background` at **ΔE 7 / 1.20:1**, is **identical across all four columns** — it is a pure luminance problem affecting normal-sighted players just as much. It is a general UI polish issue, not an accessibility one, and a colour blind mode will not touch it.

> **This is the report's most actionable finding.** The obvious instinct — "add a colour blind mode to fix the tablet" — is aimed at the wrong target. The tablet is fine. The risk is **red evidence on organic backgrounds in the 3D scene**, which no audit of `TabletTheme` can see, and which §9 addresses.

8.5 Interaction with night mode

`SettingsApplier` already owns time-of-day. Night drops ambient to ≈ RGB(0.05, 0.07, 0.13) — dark **and blue-tinted**. Two consequences:

- Low light degrades colour discrimination for everyone; combined with CVD and a dark red stain, night mode is the worst case the game can produce. A protanope — who already sees reds as abnormally dark — plus night lighting is close to a guaranteed miss.
- The blue night tint interacts with tritan correction specifically, since tritan daltonization rewrites the R and G channels heavily (see the tritan `D` matrix: `[0.7412, -0.4072, 0.6660]`).

Test the matrix of `{day, night} × {off, protan, deutan, tritan}` — eight combinations. Do not assume a filter tuned in day lighting holds at night.

---

9. What Filters Cannot Fix — and the Real Recommendation

Every accessibility standard converges on the same point, and it is not "add a filter":

- **WCAG 2.1 SC 1.4.1 (Use of Color, Level A):** colour must never be the *only* visual means of conveying information.
- **Xbox Accessibility Guideline 103:** "*Color alone should never be used to represent information.*" Anything critical expressed via colour needs at least one more signifier — shape, pattern, iconography, or text. Where colour is the primary channel, let players **choose the colours** (presets, or ideally free choice).
- **Meta VRC.Quest.Accessibility.6:** "*Applications should either provide color blindness options, or use other techniques such as combining color and pattern for easy visual distinction.*" Note the **or** — Meta accepts good design in place of a filter. It is a *recommended*, not mandatory, VRC.
- **Game Accessibility Guidelines:** "Ensure no essential information is conveyed by a colour alone" is a *basic*-tier item — the entry level, not an advanced feature.

A global daltonization filter is a **blunt** instrument: it shifts every pixel of the scene, including all the colours that were never a problem, and it cannot know that *this* red pixel is a bloodstain and *that* one is a rug pattern. It helps, but it treats the symptom.

For this project's actual risk — evidence discoverability — the higher-value work is redundant cues on `KeyEvidenceItem`:

- **Outline / rim highlight** on key evidence when looked at or when near, carrying detectability on **luminance and shape** rather than hue. There is already a hook for this: `IpadCamera.additionalExcludedLayers` is documented as being for "*custom VFX, highlights, outlines*", so a highlight layer can be stripped from photos — the highlight helps the player **find** the evidence without contaminating the **photograph**. That is a well-designed seam and it should be used.
- **Luminance contrast**, not hue contrast, in scene dressing. A stain that differs from the carpet in *brightness* is visible to every player, including achromats — the one group daltonization provably cannot help (§4.3).
- **Audio / haptic cues** on proximity to undiscovered evidence — the multi-sensory principle XAG 103 is built on, and it also serves low-vision and blind players.
- **A notebook/objective hint** naming what is still missing, since grading is pass/fail and unrecoverable.

> **Framing for the team:** the filter is the *accessibility feature*; redundant cues are the *fix*. Ship both, but if effort is constrained, the cues do more for more players — and unlike the filter, they cost nothing at pixel rate on a Quest.

---

10. Performance on Quest

The target is a Quest 3S: a tile-based mobile GPU where **bandwidth, not ALU, is the constraint**. The 3×3 multiply itself is free; reading and writing a full-screen buffer is not.

- A full-screen frame copy costs roughly **2.5 ms on an original Quest**, against a **13.8 ms** budget at 72 Hz — around **18% of the frame**. Quest 3S is considerably faster, but the ratio is the right order of magnitude to design against.
- Effects requiring extra full-screen passes (bloom, SSAO, auto-exposure, SSR) are expensive on mobile VR *specifically* because of bandwidth, not maths.

Consequences for this feature:

1. **Prefer Option B (Channel Mixer).** It folds into the existing uber post pass — no additional full-screen read/write. This is why it outranks the "proper" custom feature on this hardware.
2. **If using Option A, the pass must not run when the mode is `none`.** Return early in `AddRenderPasses` so the default path costs literally zero. Do not enqueue an identity-matrix pass.
3. **Do not add a second pass for simulation.** Compose it into the same matrix — `M` instead of `D`, same shader, same uniform.
4. **Profile on device, not in the Editor.** This project has no automated performance loop; the human runs it. Measure with the mode off and on, day and night, on the Quest.

The severity slider costs nothing: interpolate the matrix on the CPU when the setting changes and upload it as a uniform. Never interpolate per-pixel.

---

11. Recommended Roadmap

Phase 1 — Correction filter (the shippable feature)

1. Add `colorblindSeverity` and `colorblindSimulate` to `AccessibilitySettings`; validate `mode` against `none|protan|deutan|tritan` in `SetColorblindMode`, mirroring `SetMovementSpeed`.
2. Build the mode dropdown + severity slider into the Settings screen via `SettingsScreenBuilder` (the project's convention is editor builders over hand-wiring).
3. Implement `SettingsApplier.ApplyColorblindMode()`. Precompute `D = I + S(I − M)` on the CPU from interpolated Machado matrices; push one `float3x3`.
4. **Try Option B (Channel Mixer) first** — measure whether post-processing is already on. Fall back to Option A if not.
5. **Exclude the capture camera** via a dedicated Renderer asset (§8.2). Verify a photo taken with protan mode on looks identical in the gallery to one taken with it off.

Phase 2 — Developer simulation tooling 6. Dev-only simulation toggle (`Scripts/Dev/`), using `M` in the same shader path. Never exposed to players. 7. Walk both production scenes under each simulation and screenshot every piece of key evidence. **This is how the team finds the red-on-brown problems.**

Phase 3 — The real fix 8. Redundant cues on `KeyEvidenceItem` — outline/rim highlight on a dedicated layer, stripped from photos via `additionalExcludedLayers`. 9. Re-dress any evidence that fails the Phase 2 screenshots on **luminance**, not hue. 10. Consider a high-contrast mode for achromatopsia and low vision, which no filter serves.

Phase 4 — Validation 11. Test the eight `{day, night} × {mode}` combinations on device. 12. Recruit at least one colour blind tester. Simulation is a development tool and is not a substitute — Microsoft's guidance says this explicitly, and it is the difference between a feature that ships and a feature that works.

---

12. Open Questions for the Team

1. **Is post-processing currently enabled on the Quest camera?** This single fact decides Option A vs Option B, and it is a Volume/camera setting Claude cannot read from the scene.
2. **Should the correction apply to the live viewfinder?** It is a *display* surface (so: yes, corrected) but it previews a *photo* (which is stored raw). The viewfinder shows the world through the main camera's filter already — confirm this reads naturally on device rather than as a mismatch.
3. **Is there appetite for the redundant-cue work (§9),** or is the filter the whole scope of the issue? This changes the value of the feature substantially.
4. **What is the actual evidence palette?** The bloodstain/carpet question is answerable only by looking at the scenes, which needs the Editor.

---

13. References

Peer-reviewed

- Machado, G. M., Oliveira, M. M., & Fernandes, L. A. F. (2009). *A Physiologically-based Model for Simulation of Color Vision Deficiency.* IEEE Transactions on Visualization and Computer Graphics, 15(6), 1291–1298. — PDF · record
- Brettel, H., Viénot, F., & Mollon, J. D. (1997). *Computerized simulation of color appearance for dichromats.* JOSA A, 14(10), 2647–2655.
- Viénot, F., Brettel, H., & Mollon, J. D. (1999). *Digital video colourmaps for checking the legibility of displays by dichromats.* Color Research & Application, 24(4), 243–252.
- Simon-Liedtke, J. T., & Farup, I. (2016). *Evaluating color vision deficiency daltonization methods using a behavioral visual-search method.* Journal of Visual Communication and Image Representation.

Standards & platform guidance

- WCAG 2.1 — SC 1.4.1 Use of Color (Level A)
- Xbox Accessibility Guideline 103 — Expressing cues with multiple sensory methods
- Meta — VRC.Quest.Accessibility.6
- Meta Horizon OS — Accessibility design guidance

- Game Accessibility Guidelines — no essential information by colour alone
- XR Association, *Developers Guide, Chapter Three: Accessibility and Inclusive Design in Immersive Experiences*.

Algorithms & implementation

- Daltonize.org — LMS Daltonization Algorithm
- ixora.io — Color Blindness Simulation Research (documents the widespread tritanopia error confirmed in §6.4)
- colour-science — `colour.blindness.machado2009` (reference implementation; source of the matrices in §6.3)
- colorspace (R) — Color Vision Deficiency Emulation

Unity / URP

- Full Screen Pass Renderer Feature reference (Unity 6)
- Create a low-code custom post-processing effect in URP
- Daniel Ilett — Fullscreen Shader Graph in URP
- Febucci — Custom Post Processing in Unity URP

Performance

- John Austin — Fast Post-Processing on the Oculus Quest and Unity (source of the 2.5 ms / 13.8 ms figures)
- 4Experience — Post-processing for VR and mobile devices

Epidemiology

- Colour Blind Awareness — Types of Colour Blindness
- We Are Colorblind — A Quick Introduction to Color Blindness

Tools

- Color Oracle — free desktop CVD simulator, useful for checking Editor screenshots
- Colour Contrast Analyser (CCA)

Appendix A — Reproducing the Numerical Findings

The matrices, the collapse verification (§6.4), the gamut analysis (§6.4) and the `TabletTheme` audit (§8.4) were computed with three throwaway Node scripts. They depend only on the Node standard library. Re-derivation summary:

Finding	Method	Result
Daltonization collapses to one 3×3	Compare `D·rgb` against the literal stepwise pipeline over 200k random colours	max abs error **4.4×10^{-16}**
Naive tritan is unusable	Sample 32^3 RGB cube through each `D`, count values outside [0,1]	naive **73.1%** vs Machado **12.5%**
Dichromat sanity check	Inspect Viénot protan/deutan sim matrices	rows 1 and 2 identical — R,G collapse to one axis ✓
TabletTheme survives CVD	Simulate authored sRGB floats through Machado 1.0, measure WCAG contrast + CIE76 ΔE	worst meaning-carrying pair **ΔE 73**

The `TabletTheme` audit reads its palette from the literal values in `Assets/_Game/Scripts/Tablet/TabletTheme.cs`. **If the theme changes, the §8.4 conclusion must be re-checked** — its safety comes from the specific blue/amber choice, which is easy to break by accident with a well-intentioned palette tweak.

Prepared as R&D for the `accessibility.colorblindMode` setting. No production code was modified.